

Urban IntelliFlow — สรุป ฉบับเต็ม Pipeline & RAG

ทีม Urban IntelliFlow · BDI Hackathon 2026

7 มิถุนายน 2026

CONTENTS

1. บทสรุปผู้บริหาร (Executive Summary)	
2. ปัญหาและบริบท (พร้อมแหล่งอ้างอิงตัวเลข)	
3. สถาปัตยกรรมระบบ (8 ชั้น)	
4. Data Pipeline ทีละชั้น — ทำอะไร, ทำอย่างไร, ใช้เครื่องมืออะไร	
4.1 กล้อง CCTV → วิดีโอ (RTSP)	
4.2 Edge AI (Jetson) → metadata	
4.3 MQTT (Mosquitto)	
4.4 Kafka (Event Queue)	
4.5 Spark (Streaming + Batch)	
4.6 Storage	
4.7 Orchestrator + Agents (ดู §6)	
4.8 Control & Dashboards (ดู §7, §9)	
5. Edge AI — รายละเอียดและฮาร์ดแวร์	
6. Multi-Agent Orchestration	
7. RAG — ต้องทำไหม? ทำอย่างไร? (ทำแล้วในโปรโตไทป์)	
สถาปัตยกรรม RAG ที่ใช้	
เส้นทางสู่ Production (ต้องทำเพิ่ม)	
8. แหล่งข้อมูล (Data Sources) — เอาข้อมูลจากไหน	
9. Control & Actuation	
10. Frontend / UI-UX / แอปพลิเคชัน (ทำแล้ว)	

11. Infra, Security, Governance

12. Business Value — วิธีคำนวณ (โปร่งใส, ปรับได้)

13. สถานะปัจจุบัน vs สิ่งที่ต้องทำเพิ่ม

✅ ทำเสร็จแล้ว (โปรดโทป์ที่รันได้จริง)

🔧 ต้องทำเพิ่ม (Roadmap)

14. ภาคผนวก

14.1 วิธีรันเดโม

14.2 สัญญาข้อมูล Edge → MQTT (per junction / 30s)

14.3 API หลัก

14.4 Repo

1. บทสรุปผู้บริหาร (Executive Summary)

Urban IntelliFlow เปลี่ยนสัญญาณไฟจราจร *เดิม* ของเมืองขอนแก่นให้เป็นระบบอัจฉริยะ ด้วย **IoT + Edge AI** โดยไม่ทุบของเดิม — เพิ่มกล้อง CCTV, กล้อง Edge-AI (NVIDIA Jetson), และรีเลย์ Modbus ข้างตู้ควบคุมเดิม. AI นับรถแบบเรียลไทม์แล้วปรับเวลาไฟเขียว/แดง แทนตารางเวลาตายตัว.

- **ต้นทุนต่อแยก:** จาก ~2,500,000 บาท (ระบบ adaptive เต็มรูปแบบ) → **~150,000 บาท** (ลด ~94%)
- **2 โหมด:** *Automatic* (Smart PLC คู่มือแยกสำคัญ) + *Manual* (เจ้าหน้าที่กดตามคำแนะนำในแอป → ขยายทั้งเมืองในงบจำกัด)
- **แกนเทคนิค:** YOLO26 + ByteTrack (edge) → MQTT → Kafka → Spark → MongoDB/PostGIS → Multi-Agent Orchestrator → Dashboard + Control

เอกสารนี้สรุป **ทั้ง pipeline แบบละเอียด**, **ต้องทำอะไรบ้าง (ทำเสร็จแล้ว vs ต้องทำเพิ่ม)**, **เอาข้อมูลจากไหน**, และ **RAG ต้องทำไหม/อย่างไร**.

2. ปัญหาและบริบท (พร้อมแหล่งอ้างอิงตัวเลข)

ประเด็น	ตัวเลข	แหล่งข้อมูล
อันดับเมืองรถติดของไทย	อันดับ 4 (ความหนาแน่น 44.8%)	TomTom Traffic Index / รายงานจราจร
แนวโน้ม	แย่ลง +0.8 pp ต่อปี (เมืองเดียวใน Top-5)	TomTom Traffic Index
เวลาเสียต่อคน	~81 ชม./ปี ในชั่วโมงเร่งด่วน	คำนวณจากดัชนีจราจร
ทางเลือกเดิมที่แพง	LRT ~26,900 ลบ./สาย; adaptive 2.5–3 ลบ./แยก	ข้อเสนอโครงการเมือง

ข้อมูลที่ต้องเก็บเพิ่มเพื่อยืนยัน (baseline): ปริมาณจราจรจริงต่อแยก, เวลารอเฉลี่ย ก่อน/หลังติดตั้ง, ความเร็วเฉลี่ย — เก็บได้จากกล้อง YOLO26 เอง ในช่วง pilot 2–4 สัปดาห์.

3. สถาปัตยกรรมระบบ (8 ชั้น)

LAYER 1	Data Sources	CCTV(RTSP) · 3D scan · IoT sensors · GPS · Complaints · External
LAYER 2	Edge Computing	Jetson Nano/Orin: YOLO26 + ByteTrack + Re-ID (ส่งเฉพาะ metadata)
LAYER 3	Ingestion	MQTT (Mosquitto) → Kafka (partition ตามโซน, timestamp ordering)
LAYER 4	Stream/Batch	Spark Streaming (30s) + Spark Batch (รายวัน → Time-of-Day Plan)
LAYER 5	Storage	MongoDB (lake) · PostgreSQL+PostGIS (GIS/users) · MinIO (objects)
LAYER 6	Agents (Brain)	Orchestrator + 5 sub-agents + RAG Assistant
LAYER 7	Services/Apps	FastAPI/Node API · React PWA · Power BI · Officer/Citizen app
LAYER 8	Control	Smart PLC (Modbus RTU) · Officer manual · Push alerts

หลักการ **Divide & Conquer**: แต่ละโซน = 1 Kafka partition; node สรุปข้อมูลที่ชอบ ส่งเฉพาะ metadata (count, queue_length, congestion_score, timestamp, camera_id) → แบนด์วิดท์ต่ำ ทำงานได้แม้เน็ตไม่เสถียร.

4. Data Pipeline ทีละชั้น — ทำอะไร, ทำอย่างไร, ใช้เครื่องมืออะไร

4.1 กล้อง CCTV → วิดีโอ (RTSP)

- **ทำอะไร:** ดึงสตรีมวิดีโอจากกล้องที่เสียบ (1 RTSP endpoint/กล้อง)
- **ทำอย่างไร:** ใช้กล้องเดิมของเทศบาล; จุดที่ภาพไม่พอใช้ Hikvision DS-2CD2T47G2-L (ColorVu, RTSP, มองกลางคืน)
- **ข้อควรระวัง:** วิดีโอดิบ ไม่ ออกจากแยก (PDPA)

4.2 EDGE AI (JETSON) → METADATA

- **ทำอะไร:** ตรวจจับ+นับรถไม่ซ้ำต่อหน้าต่างเวลา 30 วินาที
- **ทำอย่างไร:** `ultralytics` รัน `YOLO('yolo26n.pt')` (Nano) / `yolo26m.pt` (Orin) โหมด `model.track(tracker="bytetrack.yaml")`
- **Re-ID Merge:** รวมรถซ้ำจากหลายกล้องใน node เดียว
- **Output:** payload JSON (ดูสัญญา §8)
- **โค้ดอ้างอิง:** `edge/detector.py`, `edge/publisher.py`

4.3 MQTT (MOSQUITTO)

- **ทำอะไร:** pub/sub น้ำหนักเบา node→cloud; topic `intelliflow/<zone>/<junction>`
- **ทำอย่างไร:** `paho-mqtt`, QoS 1; เปิด TLS + user/pass (ISO 27001)

4.4 KAFKA (EVENT QUEUE)

- **ทำอะไร:** คิวเหตุการณ์แบบกระจาย, จัดลำดับ timestamp, ทนทาน, fan-out ไป Spark
- **ทำอย่างไร:** bridge MQTT→Kafka; topic `intelliflow.metrics`; partition ตามโซน A/B/C
- **เครื่องมือ:** `confluent-kafka` (consumer), `bitnami/kafka` (docker)

4.5 SPARK (STREAMING + BATCH)

- **Streaming (30s):** คำนวณ `vehicle_count`, `avg_speed`, `queue_length`, `congestion_score` ต่อ node
- **Batch (รายวัน):** วิเคราะห์ย้อนหลัง → **Time-of-Day Plan** ต่อแยก; เก็บลง Hadoop/lakehouse
- **Cross-check Engine:** รวมฝั่ง supply (กล้อง) กับ demand (GPS ประชาชน + เรื่องร้องเรียน)

4.6 STORAGE

ชั้น	เทคโนโลยี	เก็บอะไร
Data Lake	MongoDB	camera events, metadata, telemetry, re-ID
Object	MinIO (S3)	วิดีโอดิบ (ชั่วคราว), point cloud, logs
Spatial/Users	PostgreSQL + PostGIS	road network, junction geometry, users, consent log
Warehouse	PostgreSQL/Lakehouse	KPI ย้อนหลัง, OD matrix, ML feature store

4.7 ORCHESTRATOR + AGENTS (ดู §6)

4.8 CONTROL & DASHBOARDS (ดู §7, §9)

5. Edge AI — รายละเอียดและฮาร์ดแวร์

- โมเดล: YOLO26n (budget) / YOLO26m (priority) — NMS-free, export ง่าย, เร็วขึ้น ~43% บน CPU, เติ้นวัตถุเล็ก (STAL) → นับ มอเตอร์ไซด์ ได้ดี (ขออนแก่นรถจักรยานยนต์เยอะ)
 - **Tracking:** ByteTrack (ID ต่อเนื่องข้ามเฟรม)
 - **ฮาร์ดแวร์แนะนำ:**
 - Jetson Nano (งบ) / Jetson Orin (แยกหลัก)
 - ทางเลือกประหยัด: Raspberry Pi 5 + Hailo-8L (~13 TOPS, ใช้กับ solar ได้)
 - กล้อง Hikvision ColorVu; LoRa gateway (RAK7258); Modbus relay (Waveshare ~3,000฿); 4G router (Teltonika RUT956); UPS LiFePO4 48V
 - **การปรับเทียบ (ต้องทำเพิ่ม):** homography ต่อกำลังเพื่อประเมินความเร็วจริง และแปลงคิเป็นเมตร
-

6. Multi-Agent Orchestration

Orchestrator Agent รับ state รวมจากทุกโซน → กระจายงานให้ 5 sub-agents (async) → ส่งผลไป Control + Dashboard

Agent	หน้าที่	เทคนิค	Output
1 Signal Timing	คำนวณเวลาไฟเขียว/แดง	rule + ML regression (จาก Spark batch)	timing_plan
2 Anomaly+Incident	จับ spike/อุบัติเหตุ	z-score (5 นาที) + Gemini บรรยายภาพ	incident_alert
3 Route Guidance	แนะนำเส้นทางประชาชน	A*/Dijkstra ถ่วง congestion (PostGIS pgRouting)	route
4 Business Value	คำนวณ ROI/เวลา/น้ำมัน/PM2.5	สูตร + assumption (ปรับได้)	metrics
5 Feedback Classifier	จัดประเภทเรื่องร้องเรียน	Gemini (ภาพ+ข้อความ)	classified_complaint

โค้ด: `backend/orchestrator.py`, `backend/agents/*.py`. โปรดักชันใช้ LangGraph/Celery+Redis ได้.

7. RAG — ต้องทำอะไร? ทำอย่างไร? (ทำแล้วในโปรโตไทป์)

ต้องทำอะไร: การ *ควบคุมสัญญาณไฟ* ไม่ต้องใช้ RAG (เป็น CV + optimization). แต่ RAG ค่อนข้างมาก สำหรับชั้น *ผู้ช่วย/อธิบาย*:

- ผู้ช่วย AI ถาม-ตอบภาษาธรรมชาติ (เจ้าหน้าที่/ประชาชน) โดยอ้างอิงข้อมูลจริง
- อธิบายเหตุการณ์โดยอิงประวัติ + นโยบาย
- จัดเส้นทางเรื่องร้องเรียนพร้อมอ้างอิงระเบียบ

สรุป: ทำ — และทำแล้ว (`backend/rag.py` , UI: `AssistantChat.jsx` , endpoint `POST /api/assistant`).

สถาปัตยกรรม RAG ที่ใช้

1. **Knowledge Base:** การ์ด FAQ ที่ดูแลเอง + chunk จาก `docs/*.md` , `CLAUDE.md` , `README` , `DEMO.md`
2. **Live State Injection:** ดึงสถานะเรียลไทม์จาก `state.py` เป็น “เอกสาร” สดทุกครั้ง → ตอบ “ตอนนี้แยกไหนรถติดสุด” ได้
3. **Retriever:** lexical hybrid — word overlap (อังกฤษ) + Thai 3-gram (ไทยไม่มีช่องว่าง) + boost คีย์เวิร์ด/แท็ก
4. **Generation:** Gemini 1.5 Flash เมื่อมี `GEMINI_API_KEY` ; ไม่มีก็ใช้ extractive (ทำงาน offline ได้)

เส้นทางสู่ PRODUCTION (ต้องทำเพิ่ม)

- เปลี่ยน retriever เป็น **embeddings + vector DB**: `pgvector` (มี Postgres อยู่แล้ว) หรือ FAISS/Qdrant
- โมเดล embedding ไทย-อังกฤษ: BGE-M3 / multilingual-e5 (self-host) หรือ Gemini embeddings
- ขยาย KB: พ.ร.บ.จราจร, SOP เทศบาล, ประวัติเหตุการณ์, metadata แยก, FAQ ประชาชน
- เพิ่ม **re-ranker** (bge-reranker) + citation + guardrail กันหลอน
- บันทึก query log เพื่อปรับปรุง (PDPA-aware)

8. แหล่งข้อมูล (Data Sources) — เอาข้อมูลจากไหน

#	แหล่งข้อมูล	ประเภท	ใช้ทำอะไร	ได้จากไหน
1	CCTV จราจรเทศบาล	วิดีโอ RTSP	นับรถ/คิว/ความเร็ว	เทศบาลนครขอนแก่น (กล้องเดิม) + เสริม Hikvision
2	ชุดข้อมูล BDI Hackathon	หลากหลาย	training/validation, baseline	BDI (โจทย์/ดาต้าเซตงาน)
3	แผนที่ถนน/แยก	Vector/GIS	road network, routing, แผนที่	OpenStreetMap Khon Kaen extract → PostGIS
4	3D City Scan	Point cloud	จำลองเลน/เรขาคณิตแยก	สแกนเมือง (ที่จัดเตรียม) / LiDAR
5	GPS ประชาชน	พิกัด+เวลา	demand-side, ETA, cross-check	แอปประชาชน (ขอ consent, anonymize)
6	เรื่องร้องเรียน/ภาพ	ข้อความ+ภาพ	จัดประเภท, RAG	แอป/สายด่วน/Traffy Fondue
7	สภาพอากาศ/สิ่งแวดล้อม	sensor/API	ปรับแผนช่วงฝน/น้ำท่วม	กรมอุตุฯ (TMD) + IoT sensor
8	PM2.5 / คุณภาพอากาศ	sensor/API	คำนวณผลด้านสิ่งแวดล้อม	Air4Thai + เซ็นเซอร์ติดเสา
9	นับจราจร/ประวัติ	ตาราง	Time-of-Day Plan, ML	เทศบาล/ขนส่งจังหวัด/กรมทางหลวง
10	ขนส่งสาธารณะ (GPS/AVL)	สตรีม	priority รถเมล์, OD	ผู้ให้บริการเดินรถ
11	เอกสารระบบ + ระเบียบ	ข้อความ	RAG KB	docs/ ในรีโป + พ.ร.บ.จราจร + SOP

หมายเหตุ: ทุกข้อมูลบุคคล (GPS/ตัวตน) ต้อง **consent + anonymize** ก่อนจัดเก็บ (PDPA).

9. Control & Actuation

- **Auto:** Orchestrator → Smart PLC → ควบคุมไฟ (แยก priority) ผ่าน Modbus RTU relay
 - **Manual:** Orchestrator → Officer app → เจ้าหน้าที่ตั้งเวลาเอง (แยกงบจำกัด) ← นวัตกรรมความคุ้มค่า
 - **Push alert:** เมื่อ congestion_score > เกณฑ์ ส่งแจ้งเจ้าหน้าที่
 - **Fail-safe (อ้างอิงแนวคิด aerospace):** watchdog heartbeat 1000ms; ถ้า AI ค้าง > เกณฑ์ → ตกไป Alternate Law = ตารางเวลาคงที่ตาม RTC (DS3231) โดย microcontroller สำรอง
-

10. Frontend / UI-UX / แอปพลิเคชัน (ทำแล้ว)

- **React PWA** (Vite) + OpenLayers (OSM ขอนแก่นจริง, ไม่ใช่ Google Maps) + recharts
 - **Auth:** Login/Register + บทบาท (ประชาชน/เจ้าหน้าที่/ผู้ดูแล), nav ตามสิทธิ์
 - **ธีม:** Light + Dark สลับได้ (จำค่า) ด้วย CSS variable tokens
 - **ฟอนต์:** Space Grotesk (display/ตัวเลข) + IBM Plex Sans Thai (UI ไทย)
 - **โลโก้:** วาง `frontend/public/logo.png` เพื่อใช้ภาพจริง (มี SVG fallback)
 - **4 ส่วน:** ภาพรวม+แผนที่+Business Value · วิเคราะห์+เหตุการณ์ · ศูนย์ควบคุม(สลับ Auto/Manual) · บริการประชาชน
 - **ผู้ช่วย AI (RAG)** ลอยมุมจอ ตอบอ้างอิงข้อมูลจริง + สถานะเรียลไทม์
 - **Power BI** สำหรับผู้บริหาร (ต้องทำเพิ่ม: เชื่อม dataset จริง)
-

11. Infra, Security, Governance

- **Containers:** Docker + Kubernetes, self-host, open-source first (เลี่ยง cloud ค่าใช้จ่ายสูง)
 - **Monitoring:** Prometheus + Grafana + Loki
 - **PDPA:** consent ก่อนเก็บข้อมูลตำแหน่ง/ตัวตน, anonymize, consent log ใน PostGIS
 - **ISO 27001:** เข้ารหัส at-rest/in-transit, RBAC, audit log
 - **Auth ปัจจุบัน:** PBKDF2 + bearer token (in-memory) → โปรตักชั้นใช้ Postgres + JWT/refresh + OAuth
-

12. Business Value — วิธีคำนวณ (โปร่งใส, ปรับได้)

ค่าคงที่อยู่ใน `backend/agents/business_value.py` (ปรับตามข้อมูลจริงได้):

- ต้นทุน: 2,500,000 → 150,000 บาท/แยก
 - เวลา/น้ำมัน/PM2.5: ประเมินจากผู้สัญจร/แยก × เวลารอที่ลด × ปัจจัยน้ำมันเดินเบา
 - ROI: เดือนที่เงินประหยัดสะสม > ต้นทุนติดตั้ง+บำรุงรักษา
 - ต้องทำเพิ่ม: แทนค่าด้วย before/after จริงจาก pilot + คาร์บอนเครดิต (TCMA)
-

13. สถานะปัจจุบัน vs สิ่งที่ต้องทำเพิ่ม

✅ ทำเสร็จแล้ว (โปรโตไทป์ที่รันได้จริง)

- Backend FastAPI + Orchestrator (สุ่มจราจรจริงแบบ rush-hour) ขับ dashboard สต ด้วยคำสั่งเดียว
- REST API ครบ (summary, junctions, mode toggle, incidents, heatmap, analytics, business-value, route, complaint, citizen, **auth**, **assistant/RAG**)
- 5 agents (rule-based) + RAG assistant + ทดสอบ pytest 8 เคสผ่าน
- Frontend: 4 แท็บ, แผนที่ซ้อนกันจริง, Business Value + ROI chart, Auth, Light/Dark, ผู้ช่วย AI
- Edge `detector.py` (YOLO26+ByteTrack) พร้อมรันกับวีดีโอตัวอย่าง
- docker-compose (Mosquitto/Kafka/Mongo/PostGIS/MinIO), CI (pytest + build)

🔧 ต้องทำเพิ่ม (ROADMAP)

ระยะ A — ต่อท่อจริง (2–3 สัปดาห์)

1. รัน YOLO26 บน Jetson จริง + ปรับเทียบ homography (ความเร็ว/คิวเป็นเมตร)
2. ต่อ MQTT→Kafka→Spark Streaming จริง (แทน simulator); สคีม่า 30s
3. เก็บลง MongoDB + PostGIS จริง; เข้า MinIO

ระยะ B — สมองและข้อมูล (3–4 สัปดาห์) 4. Spark Batch รายวัน → Time-of-Day Plan; ML regression ของ Agent 1 5. RAG production: pgvector + embeddings ไทย + re-ranker + ขยาย KB (พ.ร.บ./SOP/ประวัติ) 6. เชื่อม Gemini จริง (key) สำหรับ Agent 2/5 + ผู้ช่วย

ระยะ C — ควบคุมและนำร่อง (4–6 สัปดาห์) 7. ต่อ Smart PLC + Modbus relay ที่แยกนำร่อง; โหมด Manual จริง ให้เจ้าหน้าที่ 8. Fail-safe watchdog + RTC fallback 9. Pilot 1–3 แยก เก็บ before/after เพื่อยืนยัน ROI

ระยะ D — ขยายผลและธรรมาภิบาล 10. PDPA consent flow + audit; ISO 27001 controls; RBAC จริง (JWT) 11. K8s + Prometheus/Grafana; Power BI เชื่อม dataset จริง 12. ขยายทีละโซน (divide & conquer), จัดรูปแบบ hybrid Auto/Manual

14. ภาคผนวก

14.1 วิธีรันเดโม

```
git clone https://github.com/nathirad/Urban-intelliflow-.git
cd Urban-intelliflow-
./start-demo.sh          # backend :8000 + frontend :5173
```

บัญชีทดลอง: `officer@khonkaen.go.th` / `demo1234` (ดูเพิ่มใน DEMO.md)

14.2 สัญญาข้อมูล EDGE → MQTT (PER JUNCTION / 30S)

```
{ "node_id":"A","camera_id":"cam-001","junction_id":"MITR-01","timestamp":"ISO8601",
  "vehicle_count":42,"queue_length_m":85.0,"avg_speed_kmh":12.3,"congestion_score":0.78,
  "by_class":{"car":30,"motorcycle":10,"truck":2} }
```

14.3 API หลัก

```
/api/summary · /api/junctions · /api/junctions/{id}/mode · /api/incidents · /api/heatmap
· /api/analytics/top · /api/business-value · /api/route · /api/complaint ·
/api/citizen/stats · /api/auth/* · /api/assistant (RAG)
```

14.4 REPO

GitHub: <https://github.com/nathirad/Urban-intelliflow->

[SMARTPANTS PRESERVED 781](#)[SMARTPANTS PRESERVED 782](#)[SMARTPANTS PRESERVED 783](#)

[*Urban IntelliFlow — “Smarter Traffic, Better Khon Kaen · จราจรอัจฉริยะ เมืองน่าอยู่ ขอนแก่นยั่งยืน”*](#)